

LBT3 (Linux) Zusammenfassung Klausur Nr. 2

Lars Riemke

08.07.2026

Inhaltsverzeichnis

1	Caesar	2
2	Krypto	3
2.1	Core Principles of Cryptography	3
2.2	Symmetrische Kryptografie	4
2.2.1	VeraCrypt	4
2.3	Asymmetrische Kryptografie	6
2.3.1	Signatur vs. Zertifikat	6
2.4	Hybridverschlüsselung	7
2.4.1	Was bedeutet Hybridverschlüsselung?	7
2.4.2	Grundidee	7
2.4.3	Schritt-für-Schritt-Ablauf bei HTTPS	8
2.4.4	E-Mail Signatur und Verschlüsselung	9
3	KeePassXC	10
3.1	Installation unter Debian 13	10
4	GnuPG	11
4.1	Unterschied: Signieren vs. Verschlüsseln	11
4.2	Zertifizierungskreislauf (Web of Trust)	11
4.3	Commands	12
5	Hash	14
5.1	Kryptografische Hashfunktion	14
5.2	Wichtige Hash-Algorithmen	14
5.3	Passwörter unter Linux	14
5.4	Cracken von Hashes	14
5.5	Ablauf mit Hashcat	15
5.5.1	Hashcat-Modi	16
5.5.2	Masken für Hashcat	16
5.6	John the Ripper	17
6	SCP	18

1 Caesar

Bei einer Verschiebechiffre wird jedes Klartextzeichen z durch um ein um k Zeichen im Alphabet verschobenes Zeichen ersetzt. Bei einem Alphabet mit n Zeichen seien die Zeichen durchnummeriert von 0 bis $n-1$.

Beispiel

Shift: 3

- $A \rightarrow D$
- $B \rightarrow E$
- $C \rightarrow F$

Verschiebechiffren sind **sehr einfach zu entschlüsseln**. Ist das verwendete Alphabet zu groß, kann man versuchen den Text **statistisch zu analysieren**. Die Häufigkeit der Buchstaben variiert kaum.

Buchstabe	Häufigkeit [%]	Buchstabe	Häufigkeit [%]
a	6.51	n	9.78
b	1.89	o	2.51
c	3.06	p	0.79
d	5.08	q	0.02
e	17.40	r	7.00
f	1.66	s	7.27
g	3.01	t	6.15
h	4.76	u	4.35
i	7.55	v	0.67
j	0.27	w	1.89
k	1.21	x	0.03
l	3.44	y	0.04
m	2.53	z	1.13

Abbildung 1: Buchstabenhäufigkeit

2 Krypto

2.1 Core Principles of Cryptography

- **Geheimhaltung** (Vertraulichkeit) → Confidentiality
Unbefugte dürfen die Nachricht nicht lesen
→ **Verschlüsselung**
- **Integrität** → Integrity
Nachricht darf nicht verändert werden
→ **Signatur**
- **Authentizität** → Authenticity
Empfänger muss sicher sein, dass die Nachricht nicht von jemand anderem ist
→ **Signatur**
- **Verbindlichkeit** (Nichtabstreitbarkeit) → Non-Repudiation
Der Urheber soll nicht abstreiten können, dass er auch der Urheber ist
→ **Signatur + Zertifikat**

Merke

Die Sicherheit eines Verschlüsselungsverfahrens darf nur von der **Geheimhaltung des Schlüssels** abhängen aber niemals von der Geheimhaltung des Verschlüsselungsverfahrens.

2.2 Symmetrische Kryptografie

Secret-Key Verfahren (AES → Veracrypt)

- Nur ein Schlüssel zum Ver- und Entschlüsseln
- Vorteil: Schnell
- Nachteil: Schlüssel muss sorgfältig geheimgehalten werden
- Beispiel: WLAN-Verschlüsselung

2.2.1 VeraCrypt

VeraCrypt ist eine **Verschlüsselungssoftware**, die durch das Erstellen geschützter Container oder Laufwerke Daten vor unbefugtem Zugriff sichert. Dafür verwendet VeraCrypt unter anderem den Algorithmus **AES (Advanced Encryption Standard)**, welcher auf **symmetrischer Verschlüsselung** basiert. Das bedeutet, dass zum **Ver- und Entschlüsseln exakt derselbe Schlüssel** (Passwort) verwendet wird. Dies wird bei Festplatten und Containern eingesetzt, da symmetrische Algorithmen binär auf CPU-Ebene arbeiten, **extrem schnell** sind und daher das Lesen und Schreiben riesiger Datenmengen in Echtzeit (on-the-fly) ohne spürbare Verzögerungen ermöglichen.

1. User in sudoers-Datei einfügen

```
su
sudo visudo
<username> ALL=(ALL:ALL) ALL
```

2. Download VeraCrypt-GUI im Browser

<https://veracrypt.io/en/Downloads.html>

3. Install

```
apt install ./<filename>
```

4. Container erstellen

1. Create Volume
2. Create an encrypted file container → Next >
3. Standard VeraCrypt volume → Next >
4. Speicherort wählen → Next >
5. Encryption Algorithm: AES | KDF: HMAC-SHA-512 → Next >
6. Speicherkapazität festlegen → Next >
7. Passwort erstellen → Next >
8. Filesystem: exFAT
9. Cross-Platform Support: I will mount the volume on other platforms → Next >
10. Mauszeiger zufällig im Fenster bewegen → Format

4. Container mounten (einbinden)

1. Select File...
2. Mount
3. Passwort eingeben → OK

Merke

Wenn der VeraCrypt-Container in einer **Cloud** liegen soll muss folgende Änderung vorgenommen werden:

Settings → Preferences → Security → Preserve modification timestamp of file containers (uncheck)

Die Cloud lädt Dateiänderungen sonst nicht hoch, da sie nicht weiß, wann zuletzt an einer Datei gearbeitet wurde. Allerdings verringert das die Sicherheit da auch Unbefugte sehen können, wann die Datei zuletzt bearbeitet wurde.

2.3 Asymmetrische Kryptografie

Nutzt ein **Schlüsselpaar**, bei dem die **digitale Signatur** die Nachricht absichert und das **digitale Zertifikat** die Echtheit des Kommunikationspartners belegt.

Public-Key Verfahren (RSA → gpg)

- Öffentlicher Schlüssel nur zum Verschlüsseln
- Vorteil: Privater Schlüssel zum Entschlüsseln wird nicht getauscht
- Nachteil: Langsam (Rechenintensiv)
- Beispiel: Teil von SSL/TLS

Grundlage

- **Primfaktorzerlegung**
- Jede Zahl besteht aus einem Produkt von Primzahlen

Funktion

1. Man verteilt eine Kopie eines geöffneten Schlosses (Public-Key)
2. Möchte eine andere Person eine geheime Nachricht senden, packt sie diese in eine Kiste und verschließt diese dann mit dem Schloss.
3. Die Kiste wird verschickt. Sollte die Kiste abgefangen werden, kann sie niemand öffnen.
4. Kommt die Kiste beim Empfänger an, kann er das Schloss mit seinem Schlüssel (Private-Key) öffnen.

Jeder kann eine Nachricht für mich verschlüsseln weil jeder mein offenes Schloss (**Public-Key**) nutzen darf. Nur ich alleine kann die Nachricht wieder entschlüsseln weil ich den einzigen Schlüssel (**Private-Key**) für das Schloss habe.

2.3.1 Signatur vs. Zertifikat

- Eine digitale Signatur **sichert eine spezifische Nachricht ab**, indem ein Hashwert der Daten mit dem privaten Schlüssel des Senders verschlüsselt wird.
- Ein digitales Zertifikat ist ein **elektronischer Ausweis für den öffentlichen Schlüssel**, der die Identität des Schlüsselinhabers durch eine vertrauenswürdige Stelle zweifelsfrei bestätigt.

2.4 Hybridverschlüsselung

Grundproblem

Asymmetrische Verschlüsselung ist zwar extrem sicher, aber viel zu langsam für große Dateien.

Symmetrische Verschlüsselung ist sehr schnell aber man muss den Schlüssel erstmal sicher zum Empfänger bringen.

2.4.1 Was bedeutet Hybridverschlüsselung?

1. Asymmetrische Verschlüsselung

Wird verwendet um den **Schlüssel sicher auszutauschen oder die Identität zu prüfen**

2. Symmetrische Verschlüsselung

Wird aufgrund der Schnelligkeit für die eigentliche **Datenübertragung** verwendet.

2.4.2 Grundidee

- Wie weiß der Browser, dass der Webserver wirklich der richtige Server ist?
 - Wie können Browser und Webserver einen geheimen Schlüssel austauschen, ohne das jemand mitlesen kann?
1. Sender erzeugt auf seinem Gerät einen symmetrischen Sitzungsschlüssel (Session-Key)
 2. Der Sender verschlüsselt eine riesige Datei mit diesem Session-Key. Das geht sehr schnell.
 3. Der Sender nimmt nur den asymmetrischen Schlüssel des Empfängers und verschlüsselt damit nur den kleinen Session-Key.
 4. Datei und Sitzungsschlüssel werden dem Empfänger zugesendet. Dieser kann mit seinem Private-Key den Sitzungsschlüssel entschlüsseln und dann mit dem Sitzungsschlüssel die Datei entschlüsseln.
 5. Beispiel: https

2.4.3 Schritt-für-Schritt-Ablauf bei HTTPS

1. Der Browser ruft eine HTTPS-Website auf

Der Browser möchte eine **sichere Verbindung** aufbauen. Er teilt dem Server mit, welche **Verschlüsselungsarten und Sicherheitsverfahren** er beherrscht.

2. Der Webserver antwortet mit einem Zertifikat

Der Webserver schickt dem Browser ein **digitales Zertifikat**. In dem Zertifikat sind folgende Informationen:

1. Name der Website oder die Domain
2. Besitzer des Zertifikats
3. Aussteller des Zertifikats
4. Gültigkeitsdauer
5. öffentlichen Schlüssel des Webserver
6. Digitale Signatur der Zertifizierungsstelle (CA → Certification Authority)

3. Der Browser überprüft das Zertifikat

1. Ist das Zertifikat noch gültig?
2. Passt das Zertifikat zur Website?
3. Wurde das Zertifikat von einer Vertrauenswürdigen CA ausgestellt?

4. Die Identität des Webserver ist bestätigt

Wenn alle Prüfungen erfolgreich waren, weiß der Browser das er wirklich mit dem Webserver spricht.

5. Hybridverschlüsselung

Der Browser erzeugt einen **symmetrischen Sitzungsschlüssel (Session Key)**. Anschließend verschlüsselt der Browser den Session-Key mit dem Asymmetrischen Public-Key des Webserver und schickt diesen dann an den Webserver der diesen dann mit seinem Asymmetrischen Private Key wieder entschlüsseln kann. Dieser Session-Key ist nur für die aktuelle Verbindung gültig.

6. Sicherstellung der Datenintegrität

Ein **Message Authentication Code (MAC)** ist eine Art **Prüfsumme**, die einem geheimen Schlüssel berechnet wird (Hash-Wert). Wenn ein einziges Zeichen verändert wird, passt die Prüfsumme nicht mehr.

7. Sichere Kommunikation läuft

Die Datenübertragung läuft nun symmetrisch Verschlüsselt ab.

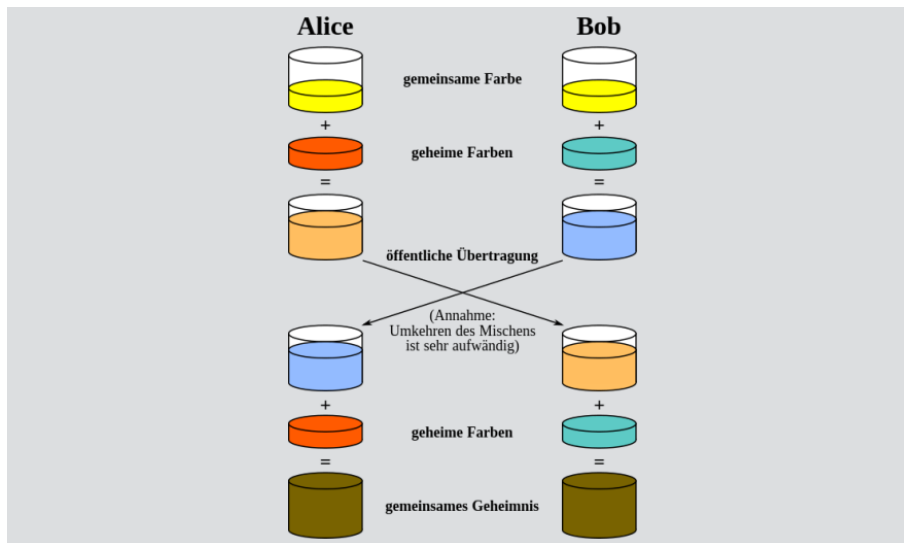


Abbildung 2: Diffie-Hellmann-Schlüsseltausch bei bspw. VPN-Verbindungen

2.4.4 E-Mail Signatur und Verschlüsselung

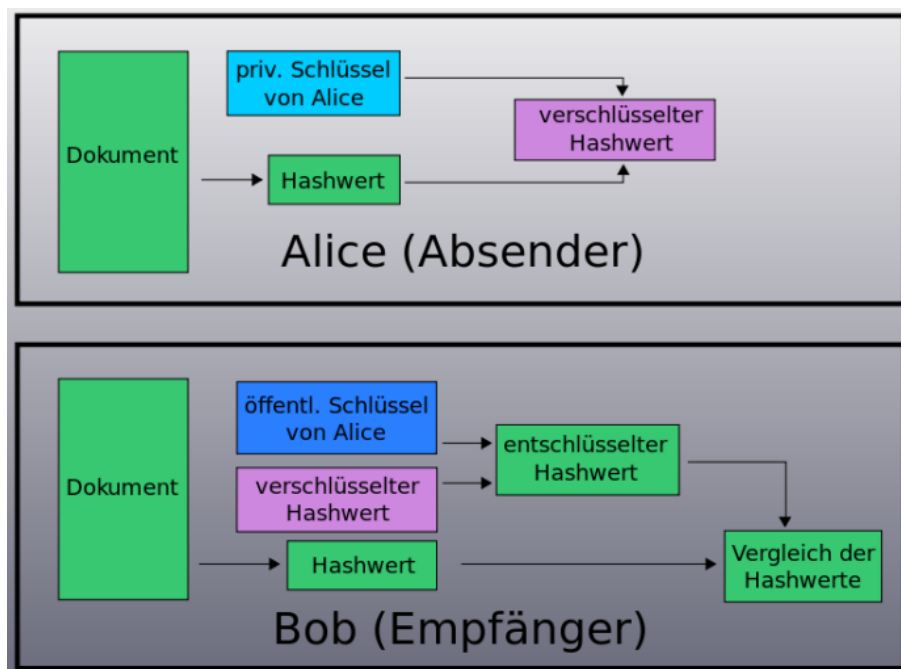


Abbildung 3: Signatur und Verschlüsselung

3 KeePassXC

Was ist KeePassXC?

KeePassXC ist ein **Passwortmanager**. Damit können Passwörter sicher in einer **verschlüsselten Datenbank** gespeichert werden. Man muss sich nur noch ein starkes **Master-Passwort** merken, mit dem die Datenbank entschlüsselt werden kann.

Warum ist ein Passwortmanager wichtig?

Passwortmanagement ist wichtig, damit man **für verschiedene Dienste starke und unterschiedliche Passwörter** verwenden kann, ohne sich alle merken zu müssen. Dadurch sinkt das Risiko, dass ein geknacktes oder wiederverwendetes Passwort direkt mehrere Konten gefährdet. Außerdem hilft KeePassXC dabei, **Zugangsdaten geordnet abzulegen**, zum Beispiel mit Titel, Benutzername, URL und Passwort.

Warum ist KeePassXC eine gute Alternative?

KeePassXC speichert Passwörter in einer eigenen verschlüsselten Datenbank. Dadurch liegen Zugangsdaten gesammelt und geschützt an einem Ort und sind nicht direkt an einen bestimmten Browser.

Welche Risiken bestehen, wenn die Datenbank schlecht gesichert ist?

Wenn das **Master-Passwort** schwach ist oder die Datenbankdatei ungeschützt abgelegt wird, könnten Unbefugte im schlimmsten Fall Zugriff auf viele gespeicherte Passwörter bekommen. Wird das Master-Passwort vergessen, kann KeePassXC es nicht zurücksetzen und die Datenbank lässt sich nicht mehr öffnen.

3.1 Installation unter Debian 13

```
sudo apt update
sudo apt install keepassxc
```

4 GnuPG

GnuPG (GNU Privacy Guard) ist ein kostenloses Open-Source-Programm zur Verschlüsselung und digitalen Signatur von Daten und Kommunikation.

1. Dateien verschlüsseln
2. E-Mails verschlüsseln
3. Digitale Signaturen erstellen
4. Schlüssel verwalten

GPG verwendet sowohl symmetrische als auch asymmetrische Verschlüsselung und kombiniert beide in einem Hybridverfahren.

4.1 Unterschied: Signieren vs. Verschlüsseln

Obwohl beide Verfahren aus einer Datei unlesbaren Zeichensalat machen, verfolgen sie unterschiedliche Ziele:

- **Signieren (Identität & Integrität):** Die Datei wird mit dem eigenen Private-Key signiert. Jeder, der den passenden Public-Key hat, kann die Datei lesen. Es gibt keinen Geheimnisschutz, aber es beweist zweifelsfrei die Herkunft und Unverändertheit der Datei (ideal für öffentliche Updates).
- **Verschlüsseln (Geheimhaltung):** Die Datei wird mit dem Public-Key des Empfängers verschlüsselt. Nur dieser kann die Datei mit seinem Private-Key wieder öffnen. Bietet absoluten Geheimnisschutz, beweist aber ohne zusätzliche Signatur nicht die Identität des Absenders.

4.2 Zertifizierungskreislauf (Web of Trust)

Damit man sicher sein kann, dass ein Public-Key wirklich zur angegebenen Person gehört, muss dieser verifiziert und zertifiziert werden:

1. **Generieren:** GPG erstellt automatisch ein selbstsigniertes Zertifikat, das Name und E-Mail an den neuen Public-Key bindet.
2. **Export & Übergabe:** Der eigene Public-Key wird exportiert und dem Partner übergeben.
3. **Verifizierung:** Der eindeutige Fingerabdruck (Fingerprint) wird auf einem sicheren Weg (z. B. persönlich oder per Video) mit dem Partner abgeglichen.
4. **Signieren (Beglaubigen):** Der Partner importiert den Schlüssel und brennt nach erfolgreichem Abgleich seine eigene digitale Unterschrift darauf.
5. **Re-Import:** Der unterschriebene Schlüssel wird zurückgeschickt und von einem selbst wieder importiert. Die fremde Zertifizierung ist nun dauerhaft im Keyring verankert.

4.3 Commands

Generieren eines Schlüsselpaares

```
gpg --full-gen-key
```

Auflisten der vorhandenen Schlüssel & Fingerprints

```
gpg --list-keys
```

→ Zeigt alle Public-Keys

```
gpg --list-secret-keys
```

→ Zeigt alle Private-Keys

```
gpg --list-keys --fingerprint
```

→ Zeigt zusätzlich den fälschungssicheren Fingerabdruck der Schlüssel

Keys löschen

```
gpg --delete-secret-key <ID>
```

→ Löscht den Private-Key

```
gpg --delete-key <ID>
```

→ Löscht den Public-Key

Im- und Export von Public-Keys

```
gpg --export -a <ID> > publickey.asc
```

```
gpg --export-secret-keys -a <ID> > privatekey.asc
```

- `-a` → `--armor` (macht den Schlüssel als ASCII-Text statt binär lesbar)

```
gpg --import publickey.asc
```

→ Lädt einen fremden Schlüssel in deinen Keyring

Dateien signieren und verifizieren

```
gpg --sign <filename>
```

→ Signiert eine Datei und macht sie unleserlich → Output: <filename>.gpg

```
gpg -d <filename>.gpg
```

→ Macht eine signierte Datei wieder lesbar

```
gpg --clearsign <filename>
```

→ Signiert die Datei so, dass der Klartext noch lesbar bleibt

```
gpg --verify <filename>.gpg
```

→ Überprüft die Echtheit einer Signatur

Verschlüsseln und Entschlüsseln

```
gpg -e -r <ID> <filename>
```

- `-e` → `-encrypt` (verschlüsselt die Datei)
- `-r` → `-recipient` (für wen wird verschlüsselt)

```
gpg -o output.txt -d datei.txt.gpg
```

- `-d` → `-decrypt` (entschlüsseln)
- `-o` → `-output` (gibt an, wo der Inhalt gespeichert werden soll)

Prüfen ob eine Verschlüsselung erfolgreich war

```
gpg --list-packets <filename>.gpg
```

Fremde Schlüssel beglaubigen (Web of Trust)

```
gpg --sign-key <ID/E-Mail>
```

→ Zertifiziert einen fremden Public-Key durch die eigene Unterschrift

```
gpg --list-sigs <ID/E-Mail>
```

→ Listet alle Unterschriften auf, mit denen ein Schlüssel zertifiziert wurde

5 Hash

5.1 Kryptografische Hashfunktion

1. Wandelt Daten (bspw. Text) in einen Hashwert um
2. Immer gleiche Eingabe → Immer gleicher Hash
3. Kleine Änderung → Kompletter anderer Hash
4. Nicht rückrechenbar

Beispiel

```
echo -n "Hallo_Welt" | sha512sum
```

5.2 Wichtige Hash-Algorithmen

1. sha1
2. sha256 / sha512 (aktuell sicher)
3. MD5 (unsicher)

5.3 Passwörter unter Linux

Passwörter werden unter linux nicht im Klartext gespeichert.

```
/etc/shadow
```

Meist wird sha512 als Verschlüsselung verwendet.

5.4 Cracken von Hashes

Tools

1. John the Ripper
2. Hashcat

Methoden

1. Brute Force (alles ausprobieren)
2. Wörterbuchangriff (Liste von Passwörtern)

5.5 Ablauf mit Hashcat

```
apt install hashcat
```

Schritt 1

Erstelle einen Hash der Telefonnummer 012345678 und speichere diesen in der Datei telefonnummer.txt

```
echo -n 0123456789 | md5sum > telefonnummer.txt
```

Schritt 2

Bereite die Datei für das Cracken vor.

```
cat telefonnummer.txt | cut -d'_' -f1 > telefonnummer_stripped.txt
```

Schritt 3

Mit welchem Algorithmus wurde der Hash erstellt?

```
hashcat --identify telefonnummer_stripped.txt
```

Schritt 4

Cracke den Hash mit Hashcat mit dem Wissen, dass die Telefonnummer 10 Zeichen hat.

```
hashcat -m 0 -a 3 telefonnummer_stripped.txt ?d?d?d?d?d?d?d?d?d?d --force
```

Schritt 5

Lasse den gecrackten Hash ausgeben.

```
hashcat -m 0 stripped.txt --show
```

Lösung in Datei schreiben

```
hashcat -m 0 -a 3 telefonnummer_stripped.txt -o loesung.txt --force
```


5.5.1 Hashcat-Modi

Algorithmus	Hashcat
MD5	-m 0
SHA1	-m 100
SHA256	-m 1400
SHA512	-m 1700

Tabelle 1: Hashcat-Modi

5.5.2 Masken für Hashcat

Maske	Bedeutung
?l	Kleinbuchstaben
?u	Großbuchstaben
?d	Zahlen
?s	Sonderzeichen
?a	Alles

Tabelle 2: Masken für Hashcat

5.6 John the Ripper

Installation

```
apt install john
```

Verwendung

```
nano ~/.bashrc
```

```
export PATH=$PATH:/usr/sbin
```

```
source ~/.bashrc
```

Gehashte Passwörter werden unter Linux in der Datei /etc/shadow gespeichert.

Passwörter cracken

```
john --format=crypt /etc/shadow
```

Cracked

```
john --format=crypt --show /etc/shadow
```

6 SCP

```
scp <file> <username>@<ip>:<path>
```

Wie kann ich bei gpg etwas runterladen und dann den schlüssel mit -verify überprüfen?